

TTNN-Backend for TinyGrad

Vraj Prajapati, Sohaib Nadeem, Brandon Rapanan



Agenda

- Problem Statement
- Step 1: Planning Using AI
- Step 2: Vibe Coding
- Models Used
- Step 3: Creating TT-NN Backend
- Results
- Conclusion and Final Takeaways



Problem Statement



Problem Statement

- A sub-1000 line Python library that implements automatic differentiation and basic neural network operations
- Want to add support for TTNN as a backend for hardware acceleration

The logo for 'tiny' is displayed in a white, pixelated, monospace-style font on a black rectangular background. The letters are composed of small squares, giving it a digital or retro aesthetic.

tinygrad: For something between [PyTorch](#) and [karpathy/micrograd](#). Maintained by [tiny corp.](#)



Step 1: Planning Using AI



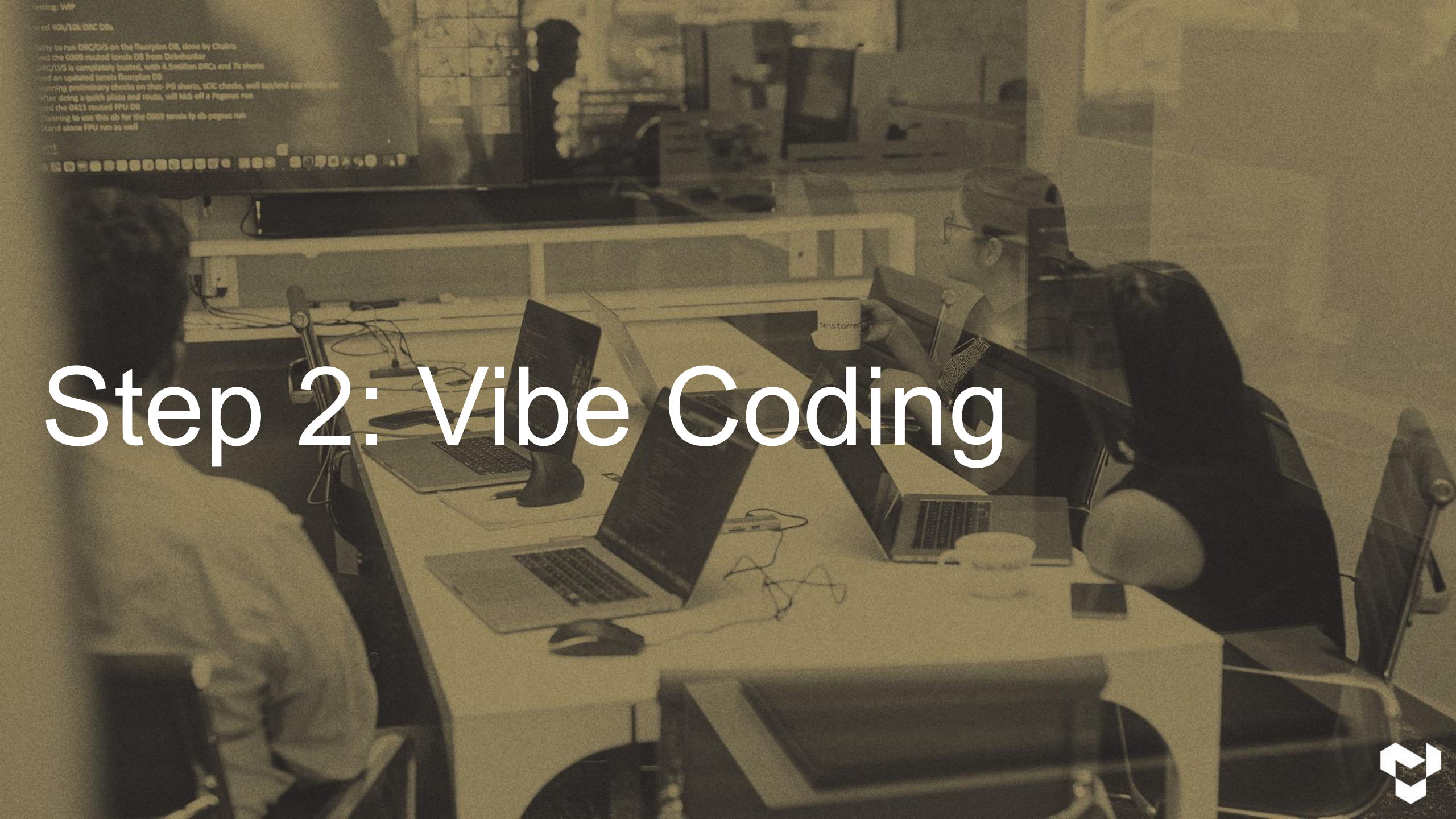
Planning Entire Hackathon project via AI

- To learn more about Tinygrad:
 - Asked claude to turn simple prompt into longer, more detailed prompt
 - Input long prompt into deep research to generate technical report
- Use Cursor IDE to scrape docs from metal repo as markdown files
- Plug in tinygrad report + documentation from metal + tinygrad to create NotebookLM
- Access the super knowledgeable TT-Tinygrad bot on NotebookLM to create a step by step guide on how to implement a TT-NN backend.
- Profit.



+





Step 2: Vibe Coding



What models to use? Benefits?

- **GPT-5**
 - Not free anymore. 1x Requests on Cursor. Good at writing code and iterating, gets stuck in loops sometimes where it tries to implement the same ideas. Definitely slow, feels like it is purposefully a little too... not verbose.
- **Sonnet 4 (thinking)**
 - Sohaib really liked it, 2x Requests. Actively talks about thoughts beyond summary so you can glean what it happening in the codebase while it is working, writes comments about this as well!
- **Sonnet 4 (not thinking)**
 - According to apple, reasoning is useless anyways! 1x Requests. Super quick changes and good for continual integration and prompting. Does not get depressed (very positive). Writes helpful comments.
 - Actually pretty useful since it doesn't clog the context with thoughts, I noticed it was a lot more "to the point" when adding large changes.

Agentic AI - Actually Pretty Useful!

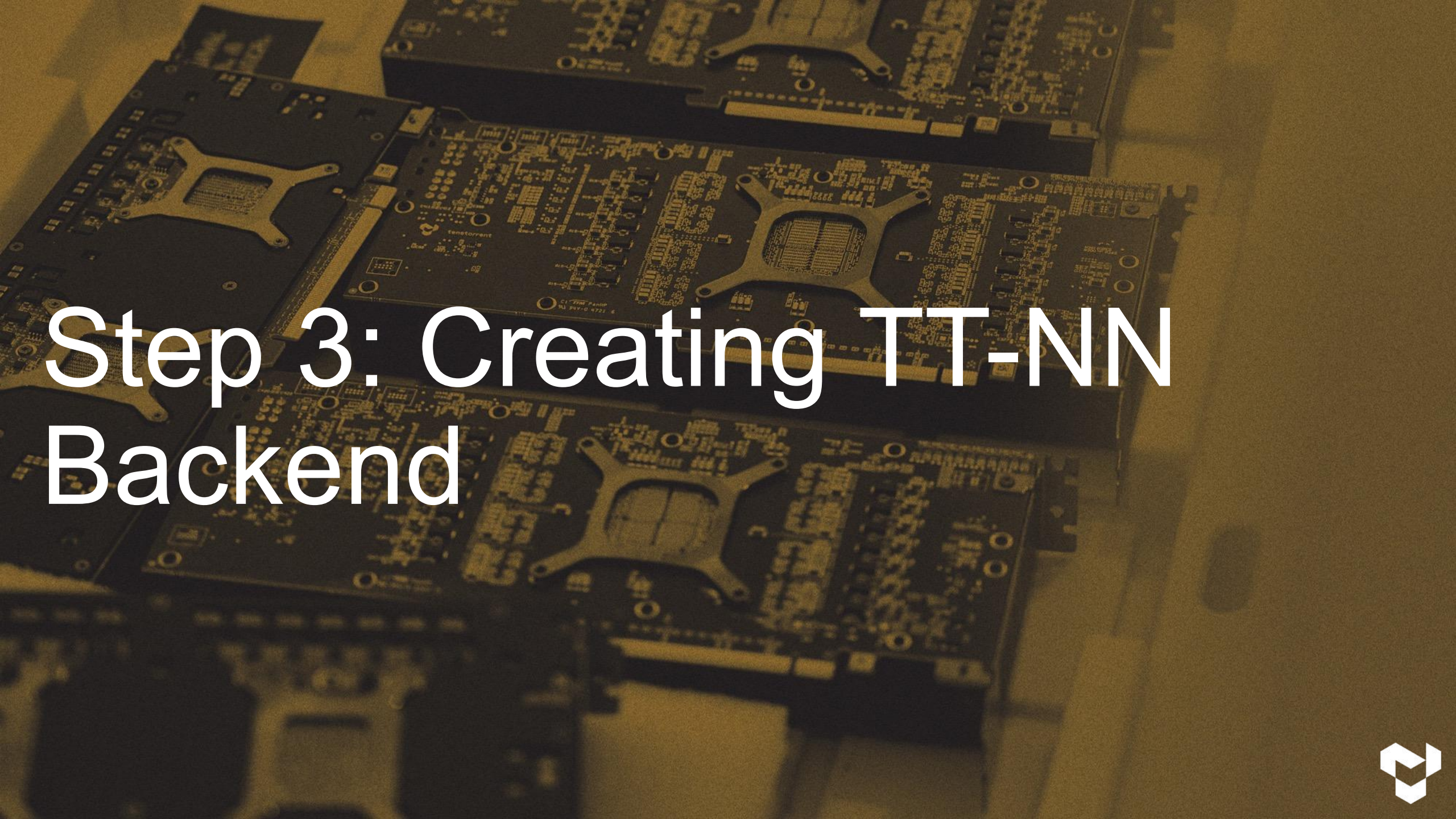
- Writing tests and telling the LLM to write tests is super beneficial to iterative development. It curbs the presence of regressions as new features are inserted.
 - Good for adding debug messages made the DevEx a lot better. More verbosity the better!
 - strategy: Tell AI to write debug statements, pipe debug messages to file and pass back into context
- Make copies of repos and plug them into your cursor so it can easily grep through the codebase!
- Todo Lists are a game changer, helps keep the LLM on track and lists out exactly what is coming in the next prompts.

The screenshot displays a development environment with three main components:

- Editor (Left):** Shows a Python file named `test_mnist_full_training.py`. The code defines a `TestMNISTFullTraining` class with a `test_full_mnist_training` method. This method performs a forward pass, calculates metrics (accuracy and loss), and includes assertions to verify training progress and inference results. A comment at the bottom states: `print("\n MNIST TRAINING COMPLETED SUCCESSFULLY!")`.
- Terminal (Bottom):** Displays the output of the script. It shows a warning about a buffer size and a series of log entries indicating the training process, including accuracy and loss values, and finally confirms "MNIST TRAINING COMPLETED SUCCESSFULLY!".
- Chat Window (Right):** Shows a conversation with an AI agent named "claude-4-sonnet". The user's prompt asks for a practical example of MNIST training with the TTNN backend. The AI's response provides a detailed explanation of the training process, including the use of `tinypgrad` and the `TTNN` backend, and includes a code snippet for setting up the environment and running the training.

Tell AI not to be stupid

- **Previous iteration of the backend was hideous spaghetti code.**
 - Be very stern and strict with the AI. Go into detail not only about where/how the spaghetti code should be refactored, but why.
 - This helps fill the context with useful information on design philosophy.
 - Take large changes step by step, leverage the todo list protocol and allow the LLM to iterate on it's changes over multiple prompts.
- **Let AI know of design contracts**
 - If you know what the structure of your code should look like, inform the AI and let it design around this. Makes the discovery space smaller and focuses the responses and edits in the code.
 - We told the AI we wanted to use a JSON “program factory” to execute the TTNN ops in the program, it followed that idea quite well.

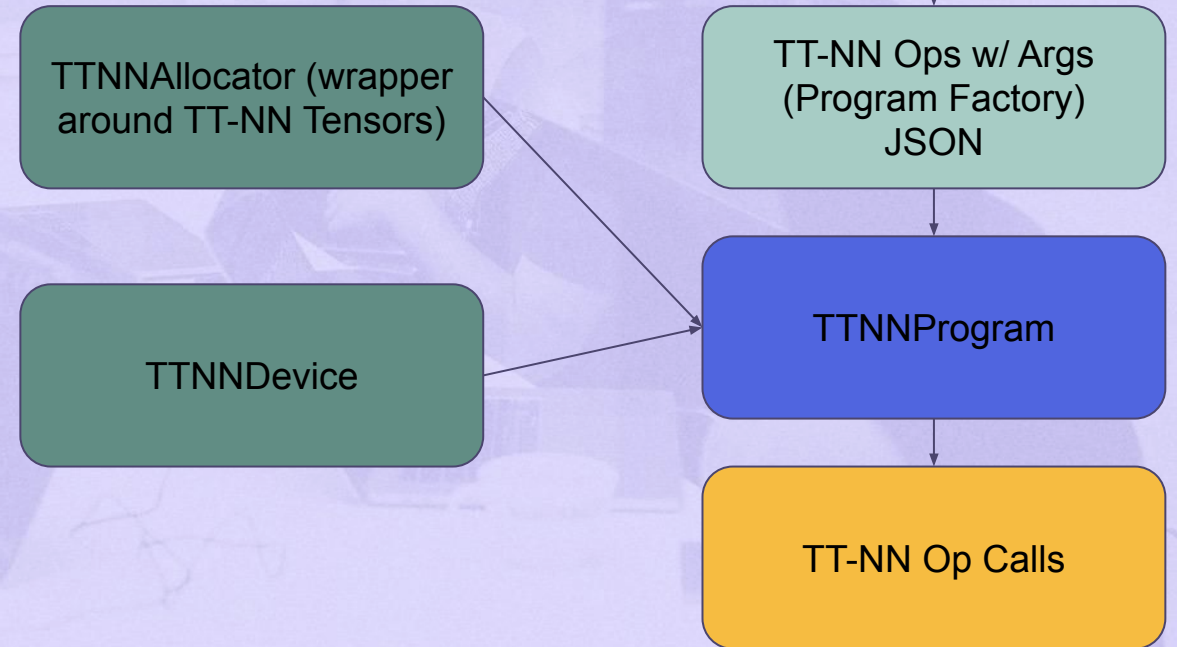


Step 3: Creating TT-NN Backend



Introducing the TT-NN Backend for Tinygrad

- General skeleton structure was inspired by the planning document created initially.
- 2 challenging questions emerged:
 - Is the backend we are going to use TT-MLIR or TT-NN
 - How do we want to turn UOps (tinygrad IR) into Ops for these formats.
- TT-NN was chosen for the simplicity + the lack of exposed runtime bindings on the tt-mlir front.
- We decided to make a “program factory” in JSON that would essentially house a list of TT-NN ops to run with their arguments.



Lowering UOps (TTNNRenderer)

- **STORE/LOAD/INDEX/SPECIAL/GPE/WhateverWhatever**
 - Due to the GPU-like nature of most TinyGrad backends, a lot of the IR pertains to how to represent threads, pools, blocks and other core GPU programming concepts.
 - This isn't necessary trivial to implement with a high level TT-NN library but it was vibe coded away into a series of python dictionaries that store local/global context variables which are used in the execution process.
- **Unary/Binary Ops**
 - Implemented these as ttnn Op calls (Add -> ttnn.add)
 - Work pretty well, they operate on TT-NN tensors but the lazy evaluation means that tensors are moved to device right when the result of some computation is to be "realized".
- **Matmul???**
 - Only Warp Matrix Multiplication (WMMA) is natively supported, and this requires a custom definition of tensor cores and the whole lot to define in tinygrad. Native support for non-gpu type accelerators is therefore difficult, but accomplishable over time.



Results



Feature List

- Interface with BFloat16 and multiple datatypes!
- Run arithmetic operations with TTNN devices, supports multicore :)
 - 10 million param Eltwise Add on Multicore
- All the basic EltWise ops for backwards and forward graph passes.
- Tinygrad integration and utility of TT-NN wheels for python-only dev environment.

Results

Comparing TTNN-Backend with NumPy:

- elementwise ops are working but perf is terrible sadly for a couple of reasons:
 - overhead for device initialization and copying tensors to device
 - we only tested in relatively small tensor sizes (100k)
 - op graph is still very unoptimized

Last second new benchmarks :)
(amortizing runtime by running 10 iterations of eltwise add with 1M element tensors)



Quick Benchmark Comparison: Addition vs Pointwise Multiply

Results Summary

Operation	NumPy Avg Throughput	TTNN Avg Throughput	NumPy Advantage
Addition	2,561 M elements/sec	1.55 M elements/sec	1,649x faster
Pointwise Multiply	2,775 M elements/sec	1.57 M elements/sec	1,770x faster



Pure Computation Results: The Real Story

Pure Performance Summary (1M Elements, No Overhead)

Operation	NumPy (M/sec)	TTNN (M/sec)	NumPy Advantage	Best Case
Addition	1,395.26	278.81	5.0x faster	13.9x faster
Multiplication	1,893.08	20.71	91.4x faster	12.3x faster

MNIST Inference + Training

- Even with the incredibly slow matmuls, it didn't stop us from writing a plethora of tests to integrate and test with tinygrad.
 - Matmuls are executed in loops, with a huge number of indexing ops (Where) alongside the primitive computations (add, multiply). Obviously this may work well for CPU backends but not for us.
- In the end we were able to test that mnist inference (without weights) works pretty slowly, and training is so slow you'd think you were training an LLM.
- There are many optimizations to be made in the future, but I think this is a good starting point.
- There is significant device time overhead and maintenance in tinygrad, we noticed that tinygrad took a long time doing setup between ops, which has caused issues for us.

Conclusion / Final Takeaway



tenstorrent

G A L A X Y



TinyGrad - Review

- Despite the extensible nature advertised, it's clear that they have truly only run with GPU workloads in intense testing.
- Would be best to implement this with the TT-MLIR compiler and D2M workflow. If we can dynamically generate kernels it would optimize the reader/writer kernels and give us less redundant DRAM writes, especially in the matmul computations.
- TinyGrad is overall a cool project, and adding new backends is actually much easier than we previously thought. It would be good to adopt this open philosophy and technology into our tech stack.

Developing with AI in the Long Term

- Excellent for just creating large datasets of knowledge and high level planning. It's an amazing tool when used in collaboration with strong developer practice.
- Fine point editing with AI has come a long way, but the integration with complex projects leaves much to be desired. While it can grep through the project it has a hard time knowing where to look for stuff. Maybe this will become less of a problem with the proliferation of [Agents.md](#) files (something we should have done).
- Iteration is key, as long as your context is full and populated there is a lot more strength to your LLM (as long as the context isn't too sticky *cough* GPT *cough*)
- The more domain knowledge you have in a subject the more useful the tools will be
 - Easier to validate generated code with a glance



Check out our Project!

<https://github.com/sohaibnadeemTT/tinygrad>

Branches: vprajapati/vibe-mnist,
snadeem/ttnn-backend

